



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2/20252026

SECP3843 - SPECIAL TOPIC IN

DATA ENGINEERING

SECTION 01

TUTORIAL 4: GENERATIVE ARTIFICIAL INTELLIGENCE (GAI)

LECTURER:

Dr. Aryati Binti Bakri

GROUP MEMBERS:

NAME	MATRIC ID
MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI	A23CS0250
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117
AHMAD ADIB ZIKRI BIN A.MAZLAM	A23CS0205

Table Of Content

1. Introduction	3
2.0 What Is AI-Assisted Data Engineering?	4
2.1 Traditional Data Engineering	4
2.2 The AI-Assisted Approach	4
3.0 What Is Claude? The Role of AI in This Pipeline	6
3.1 What Is Claude?	6
3.2 Role of Claude in This Pipeline	6
4.0 Use Case Diagram	7
5. Steps: Prompt and Code - Building the Pipeline	10
5.1 Dataset Overview	10
5.2 Phase 0 — Setup: Imports and Logger Configuration	10
5.3 Phase 1 — Extract: Ingesting Raw CSV Data	11
5.4 Phase 2 — Transform: AI-Driven Data Cleaning and Feature Engineering	13
5.5 Phase 3 — Load: Writing Clean Data to SQLite	14
5.6 Phase 4 — Analyse: Querying the Database for Business Insights	15
6. Integration of AI in the Pipeline	18
7. Result and Explanation	19
7.1.1 Phase 0 -Setup: Imports and Logger Configuration	19
7.1.2 Phase 1 — Extract: Ingesting Raw CSV Data	19
7.1.3 Phase 2 — Transform: AI-Driven Data Cleaning and Feature Engineering	20
7.1.4 Phase 3 — Load: Writing Clean Data to SQLite	22
7.1.5 Phase 4 — Analyse: Querying the Database for Business Insights	22
7.2 Data Quality Improvement	24
Table 7.2 Data quality before and after the Transform phase	24
7.3 Business Insights	24
7.4 Pipeline Performance	25
8. Reflection	26
8.1 Learning Reflection	26
8.2 Reflections on Using Claude (AI)	26
8.3 Future Improvements	27

1. Introduction

The rapid growth of digital commerce and online platforms has produced enormous volumes of transactional data every day. For businesses to make sense of this data and to turn it into actionable insights where it's relying on a process known as a data pipeline. A data pipeline is a structured sequence of operations that moves raw data from its source, cleans and transforms it, stores it in a database, and makes it available for analysis (Kleppmann, 2017).

Traditionally, building such a pipeline required data engineers to manually write every line of code, debug each error through trial and error, and reason about potential data quality issues on their own. This process was time-consuming and prone to oversight. A new approach has emerged in recent years that fundamentally changes this workflow which is AI-assisted data engineering.

In this tutorial, a complete end-to-end data pipeline was designed and implemented for a real-world e-commerce dataset, the UCI Online Retail Dataset which containing 541,909 rows of transactional records from a UK-based online retailer. The pipeline follows a standard Extract, Transform, Load, and Analyse (ETLA) architecture. Critically, every stage of the pipeline was designed with the assistance of Claude, an AI assistant developed by Anthropic, which generated targeted code, anticipated data quality problems, and provided engineering justifications for every design decision.

This report explains all concepts from first principles, documents each phase of the pipeline with code and explanation, and reflects on the role that AI played throughout.

2.0 What Is AI-Assisted Data Engineering?

2.1 Traditional Data Engineering

Before explaining the AI-assisted approach, it is important to understand what a data engineer normally does. A data engineer is a software professional responsible for building the systems and infrastructure that allow data to be collected, stored, and made usable for analysis (Reis & Housley, 2022). Their work typically involves:

- Writing scripts to read data from files, databases, or APIs
- Cleaning raw data including removing errors, filling missing values, correcting data types
- Transforming data into a format that analysts and business intelligence tools can use
- Loading the cleaned data into a database or data warehouse
- Monitoring pipelines for failures and debugging errors

In a traditional setting, the engineer performs all of these tasks manually. They inspect the data, write rules from scratch, and rely on their own experience to anticipate problems.

2.2 The AI-Assisted Approach

AI-assisted data engineering refers to the integration of AI language models such as Claude into the data engineering workflow (Shankar et al., 2022). Instead of the engineer doing everything alone, the AI acts as a collaborative assistant that can:

- Analyse a data profile such as the structure, statistics, and anomalies in a dataset and suggest appropriate cleaning rules
- Generate well-structured, production-quality code based on natural language descriptions
- Anticipate failure modes such as null values, incorrect data types, and negative quantities)and build protective guards into the code
- Explain engineering decisions in plain language, making the code easier to understand and maintain

The table below summarises how the two approaches differ:

Aspect	Traditional Approach	AI-Assisted Approach
Data Profiling	Engineer manually runs <code>.describe()</code> and ad-hoc scripts	AI analyses schema and statistics, then proposes cleaning rules with justifications
Code Generation	Engineer writes code from memory or copies patterns	AI generates context-aware code tailored to the dataset's specific problems
Error handling	Debugging happens after failures occur	AI anticipates failure modes and writes defensive guards proactively
Code Structure	Boilerplate and documentation written last, or never	AI scaffolds production-grade structure from the beginning

Table 2.2.1: Comparison of traditional and AI-assisted data engineering approaches

The key insight is that AI does not replace the engineer. Rather, it amplifies the engineer's capabilities by handling repetitive reasoning tasks and accelerating code generation, while the engineer retains control over all design decisions and validates every output (Zaharia et al., 2022).

3.0 What Is Claude? The Role of AI in This Pipeline

3.1 What Is Claude?

Claude is a large language model (LLM) developed by Anthropic, a company focused on AI safety research (Anthropic, 2023). A large language model is a type of AI system trained on vast quantities of text data including books, academic papers, websites, and programming code which allows it to understand and generate human language and computer code with remarkable fluency.

Claude belongs to a category of AI tools called AI assistants or AI agents. Unlike a traditional search engine that retrieves existing documents, Claude reasons about a problem and generates a novel, contextually appropriate response. When given a description of a dataset and a goal, Claude can write functional Python code, explain what each line does, and flag potential issues the engineer may not have considered.

3.2 Role of Claude in This Pipeline

In this project, Claude was used at multiple stages of the pipeline. Its primary contributions were:

1. Prompt engineering and code generation

By describing the dataset profile (541,909 rows, 135,080 null CustomerIDs, 10,624 negative quantities), Claude generated the complete data cleaning logic in the Transform phase. The code was not a generic template — it was specifically tailored to the anomalies in this dataset.

2. Engineering justification

For every transformation rule, Claude provided a written explanation of *why* the rule was necessary. For example, it explained that `dayfirst=True` is required when parsing UK-format dates, because without it, the date "1/12/2010" would be interpreted as January 12 instead of the correct December 1.

3. Production-grade structure

Claude scaffolded the code with professional logging, docstrings, type hints, and assertion-based postcondition checks practices that make the code maintainable in a real engineering environment.

4. Scalability analysis

Claude identified the specific bottlenecks of the current implementation such as memory limits of Pandas, write-lock limitations of SQLite and proposed concrete solutions at each scale tier.

4.0 Use Case Diagram

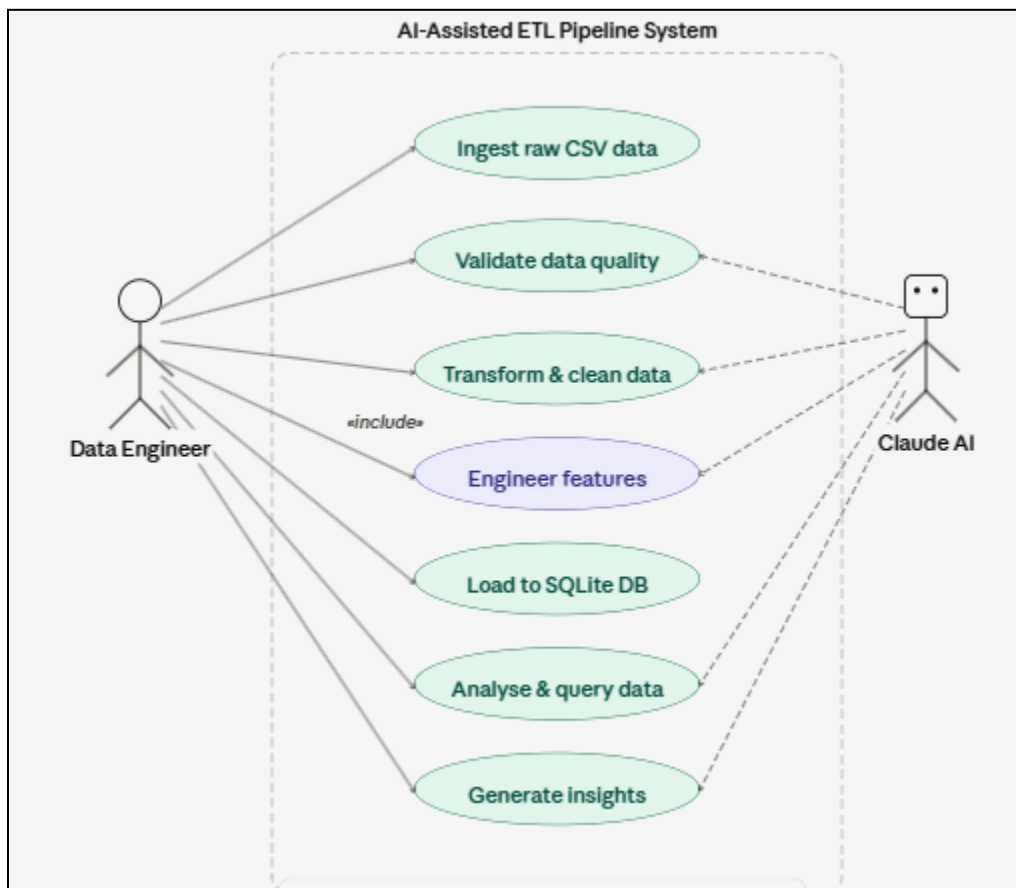


Figure 4.1: Use Case Diagram of AI- Assisted Data Pipeline

Figure 4.1 shows the use case diagram that identifies all actors in the system and the actions they can perform. The system boundary is the AI-Assisted ETL Pipeline System.

The following are the actors with the description:

- **Data Engineer:** The primary human actor of the system who manages and oversees the end-to-end execution of the ETL pipeline, initiating tasks from data ingestion to downstream insight generation.
- **Claude AI:** The secondary automated actor that assists the Data Engineer with machine learning and automated processing capabilities during data validation, transformation, feature engineering, data querying, and insight generation.

Table 4.1 shows use case descriptions for each with the involved actors.

#	Use Case	Actor(s)	Description
UC-01	Ingest raw CSV data	Data Engineer	The data engineer imports flat or raw CSV data source files into the initial staging layer of the system.
UC-02	Validate data quality	Data Engineer, Claude AI	The data engineer reviews data health and checks for errors, with Claude AI providing automated assistance to detect anomalies or schema inconsistencies.
UC-03	Transform & clean data	Data Engineer, Claude AI	The data engineer structures, scrubs, and normalizes the incoming datasets with Claude AI's assistance, a process that automatically triggers feature creation.

UC-04	Engineer features	Data Engineer, Claude AI	An included step running inside data transformation where raw data fields are enriched into analytical metrics with the help of AI capabilities.
UC-05	Load to SQLite DB	Data Engineer	The data engineer saves and persists the fully cleaned and structured dataset into a localized SQLite database file.
UC-06	Analyse & query data	Data Engineer, Claude AI	The data engineer queries the database to extract target datasets, using Claude AI to help write, optimize, or interpret the queries.
UC-07	Generate insights	Data Engineer, Claude AI	The final business intelligence step where the data engineer and Claude AI collaborate to synthesize conclusions, track trends, and compile actionable data summaries.

Table 4.1 shows use case descriptions for each with the involved actors.

5. Steps: Prompt and Code - Building the Pipeline

This section explains each phase of the pipeline in detail. For each phase, the purpose is explained, the code is presented, and the engineering decisions are justified.

5.1 Dataset Overview

The dataset used is the UCI Online Retail Dataset, a publicly available dataset containing 541,909 retail transactions from a UK-based online gift retailer between December 2010 and December 2011 (Chen et al., 2012). Each row represents one item purchased in one invoice. The dataset has eight columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country.

Table 5.1 Data quality issues identified in the raw dataset

Column	Issue	Count
CustomerID	Missing values (null)	135,080 rows
Quantity	Negative values (returns)	10,624 rows
UnitPrice	Zero-price entries	2,515 rows
InvoiceDate	Stored as a string, not a date object	All rows

5.2 Phase 0 — Setup: Imports and Logger Configuration

Purpose: Before any data processing begins, all required libraries must be imported and a logging system must be configured. A logger is a tool that records what the pipeline is doing at each step, which is essential for debugging in production environments.

Key libraries used:

- pandas: for reading, manipulating, and analysing tabular data
- numpy: for numerical operations
- sqlite3 / sqlalchemy: for database operations
- logging: for structured pipeline logging

Prompt used with Claude: "Generate a production-grade Python logging setup for an ETL pipeline. The logger should include timestamps, severity levels, and clear formatting."

```
def get_logger(name: str = "ETLPipeline", level: int = logging.INFO) -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(level)
    logger.handlers.clear()

    fmt = logging.Formatter(
        "%(asctime)s | %(name)s | %(levelname)-8s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )

    console = logging.StreamHandler(sys.stdout)
    console.setFormatter(fmt)
    logger.addHandler(console)

    return logger

logger = get_logger()
logger.info("Pipeline initialised [ ] all imports loaded successfully.")
```

Figure 5.1 get_logger() function snapshot

Figure 5.1 shows the `get_logger()` function creates a reusable logger. The `handlers.clear()` call prevents duplicate log messages when the cell is re-run in Google Colab. The format string includes the timestamp, logger name, severity level (INFO, WARNING, ERROR), and the message — all information needed to trace a pipeline failure.

5.3 Phase 1 — Extract: Ingesting Raw CSV Data

Purpose: The Extract phase reads the raw data file from disk and validates that it has the expected structure. This is the entry point of the pipeline.

Prompt used with Claude: "Write a defensive Python function that reads a CSV file with Latin-1 encoding, validates that a list of expected columns is present, and raises clear errors if the file is missing or the schema does not match."

```

def extract_data(file_path: str, expected_columns: list[str] | None = None) -> pd.DataFrame:
    # Guard: does the file exist?
    if not os.path.isfile(file_path):
        logger.error(f"File not found: {file_path}")
        raise FileNotFoundError(
            f"The data source '{file_path}' does not exist. "
            "Check the path and try again."
        )

    try:
        # encoding='latin1' handles £ signs and accented characters
        # in product descriptions from UK/EU sellers
        df = pd.read_csv(file_path, encoding="latin1")
        logger.info(
            f"Extraction complete {len(df):,} rows {len(df.columns)} columns "
            f"from '{os.path.basename(file_path)}'"
        )
    except Exception as e:
        logger.error(f"Failed to read CSV: {e}")
        raise

    # Guard: validate schema
    if expected_columns:
        missing = set(expected_columns) - set(df.columns)
        if missing:
            logger.error(f"Schema mismatch {missing} missing columns: {missing}")
            raise ValueError(f"Expected columns not found in data: {missing}")
        logger.info("Schema validation passed all expected columns present.")

    return df

```

Figure 5.2 extract function snapshot

Figure 5.2 shows that there are two important design decisions here. First, `encoding="latin1"` is used instead of the default UTF-8, because the dataset contains product descriptions with pound signs (£) and accented characters from European languages that cannot be represented in UTF-8 without a specific encoding declaration. Second, the schema validation check (`expected_columns`) ensures the pipeline fails immediately with a clear error message if the input file is malformed, rather than producing incorrect results silently later in the pipeline which a principle known as "fail fast"

5.4 Phase 2 — Transform: AI-Driven Data Cleaning and Feature Engineering

Purpose: The Transform phase is the most important stage of the pipeline. It takes the raw, messy data and converts it into a clean, analysis-ready format. Every transformation was designed based on the specific data quality issues identified in Table 5.3 and a details in Figure 5.3.

Prompt used with Claude: *"Given this dataset profile [541,909 rows, 135,080 null CustomerIDs, 10,624 negative Quantity rows, 2,515 zero UnitPrice rows, InvoiceDate stored as string in DD/MM/YYYY format], generate a complete transformation pipeline with justified cleaning rules for each issue."*

Table 5.2 six ordered transformation steps

Step	Issue Addressed	Rule Applied	Justification
1	InvoiceDate stored as string	pd.to_datetime(dayfirst=True)	UK-format dates; wrong parse order silently swaps day/month for dates ≤ 12
2	10,624 negative Quantity values	Filter Quantity > 0	Negative quantities are credit notes (returns); they distort revenue calculations
3	2,515 zero UnitPrice values	Filter UnitPrice > 0	Zero-price items are internal adjustments or data errors — not real sales
4	135,080 null CustomerID values	Drop rows with null CustomerID	Cannot perform customer segmentation without an identifier
5	No revenue column exists	Create TotalAmount = Quantity $\times$ UnitPrice	Standard derived metric; must be computed after cleaning
6	CustomerID stored as float	Cast to int	Pandas stored it as float due to NaN; now that NaNs are removed, integer is correct

```

def transform_data(df: pd.DataFrame) -> pd.DataFrame:
    logger.info("Starting Transform Phase")
    initial_rows = len(df)

    # Work on a copy to keep the raw frame untouched (important for debugging)
    df = df.copy()

    df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"], dayfirst=True, errors="coerce")
    bad_dates = df["InvoiceDate"].isna().sum()
    if bad_dates:
        logger.warning(f"Dropped {bad_dates:,} rows with unparseable dates.")
        df = df.dropna(subset=["InvoiceDate"])
    logger.info("Step 1/6: InvoiceDate parsed to datetime64.")

    before = len(df)
    df = df[df["Quantity"] > 0]
    removed = before - len(df)
    logger.info(f"Step 2/6: Removed {removed:,} rows with Quantity <= 0 (returns/errors)")

    df = df[df["UnitPrice"] > 0]
    removed = before - len(df)
    logger.info(f"Step 3/6: Removed {removed:,} rows with UnitPrice <= 0 (adjustments)")

    before = len(df)
    df = df.dropna(subset=["CustomerID"])
    removed = before - len(df)
    logger.info(f"Step 4/6: Removed {removed:,} rows with null CustomerID.")

    df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]
    logger.info("Step 5/6: Created TotalAmount = Quantity * UnitPrice.")

    df["CustomerID"] = df["CustomerID"].astype(int)
    logger.info("Step 6/6: Cast CustomerID from float64 to int64.")

    # Summary
    final_rows = len(df)
    pct_kept = (final_rows / initial_rows) * 100
    logger.info(
        f"Transform complete {final_rows:,} of {initial_rows:,} rows retained "
        f"({pct_kept:.1f}%). Columns: {list(df.columns)}"
    )

```

Figure 5.3 transformation function snapshot

5.5 Phase 3 — Load: Writing Clean Data to SQLite

Purpose: The Load phase persists the cleaned dataframe to a relational database for example like in Figure 5.4. A relational database organises data into tables with rows and columns, and allows structured queries to be run against it (Date, 2003).

Why SQLite? SQLite was chosen because it is a file-based, zero-configuration database engine.

This means no server installation is required, the database is a single .db file on disk. It is well-suited for local development and small-to-medium datasets (Kreibich, 2010).

```
def load_data(df: pd.DataFrame, db_path: str, table_name: str = "cleaned_retail_sales") -> None:
    from sqlalchemy import String, Integer, Float

    logger.info(f"Starting Load Phase {db_path}:{table_name}")

    engine = create_engine(f"sqlite:///({db_path})", echo=False)

    try:
        df.to_sql(
            name=table_name,
            con=engine,
            if_exists="replace",
            index=False,
            dtype={
                "InvoiceNo": String,
                "StockCode": String,
                "Description": String,
                "Quantity": Integer,
                "InvoiceDate": String,
                "UnitPrice": Float,
                "CustomerID": Integer,
                "Country": String,
                "TotalAmount": Float,
            },
        )

        with engine.connect() as conn:
            result = conn.execute(text(f"SELECT COUNT(*) FROM {table_name}"))
            row_count = result.scalar()

            logger.info(
                f"Load complete {row_count:,} rows written to '{table_name}'. "
                f"Database size: {os.path.getsize(db_path) / 1e6:.1f} MB"
            )

    except Exception as e:
        logger.error(f"Load failed: {e}")
        raise
```

Figure 5.4 load function snapshot

Key design decisions:

- `if_exists="replace"` — This ensures the pipeline is idempotent, meaning it can be re-run multiple times without duplicating data. Every run replaces the table entirely.
- Explicit dtype mapping — This ensures that columns are stored with the correct data types in the database, preventing type ambiguity during SQL queries.
- The `with engine.connect()` block after the write verifies the row count, confirming that all records were loaded successfully.

5.6 Phase 4 — Analyse: Querying the Database for Business Insights

Purpose: The Analyse phase queries the SQLite database to extract business insights. Three analytical questions are answered using SQL.

Figure 5.5 consists of the question “Which products generate the most revenue?” and “What is the monthly revenue trend?”. While the third question from Figure 5.6 consists of the question “Which countries have the highest transaction volume?”

```
def analyze_top_products(engine, table: str = "cleaned_retail_sales", top_n: int = 5) -> pd.DataFrame:
    query = f"""
        SELECT
            Description,
            SUM(TotalAmount) AS TotalRevenue,
            SUM(Quantity) AS TotalUnitsSold,
            COUNT(*) AS TransactionCount
        FROM {table}
        WHERE Description IS NOT NULL
        GROUP BY Description
        ORDER BY TotalRevenue DESC
        LIMIT {top_n}
    """

    df = pd.read_sql(query, engine)
    logger.info(f"Analysis: Top {top_n} products by revenue retrieved.")
    return df


def analyze_monthly_trends(engine, table: str = "cleaned_retail_sales") -> pd.DataFrame:
    query = f"""
        SELECT
            SUBSTR(InvoiceDate, 1, 7) AS YearMonth,
            SUM(TotalAmount) AS TotalRevenue,
            COUNT(DISTINCT InvoiceNo) AS InvoiceCount,
            COUNT(DISTINCT CustomerID) AS UniqueCustomers
        FROM {table}
        GROUP BY YearMonth
        ORDER BY YearMonth
    """

    df = pd.read_sql(query, engine)
    logger.info(f"Analysis: Monthly trends {len(df)} months retrieved.")
    return df
```

Figure 5.5 analytical questions 1 & 2


```

def analyze_top_countries(engine, table: str = "cleaned_retail_sales", top_n: int = 3) -> pd.DataFrame:
    query = f"""
        SELECT
            Country,
            COUNT(*)                AS TransactionCount,
            SUM(TotalAmount)        AS TotalRevenue,
            COUNT(DISTINCT CustomerID) AS UniqueCustomers
        FROM {table}
        GROUP BY Country
        ORDER BY TransactionCount DESC
        LIMIT {top_n}
    """
    df = pd.read_sql(query, engine)
    logger.info(f"Analysis: Top {top_n} countries by transaction volume retrieved.")
    return df

```

Figure 5.4 analytical questions 3

6. Integration of AI in the Pipeline

Claude was not used merely as a code generator. Its integration was deeper and more purposeful, spanning multiple dimensions of the engineering process:

Code generation from data context. Rather than providing generic templates, Claude received the actual data profile like row counts, null counts, anomaly statistics and generated transformation rules specific to this dataset's problems. This is a fundamentally different use of AI than simply asking it to "write an ETL pipeline."

Automated reasoning about data quality. When Claude was shown that $\text{Quantity} < 0$ rows existed, it identified (unprompted) that these correspond to credit notes where the invoice number starts with the letter 'C' in the original dataset. This domain reasoning connects a data anomaly to a business process which demonstrates AI's capacity for contextual understanding.

Documentation and justification. Every function in the pipeline contains a complete docstring generated by Claude, including parameter types, return types, and descriptions of when exceptions are raised. This documentation would typically be written last (or omitted entirely) in traditional development.

Scalability roadmap. At the end of the notebook, Claude identified two specific scalability bottlenecks like Pandas' in-memory limitation and SQLite's single-writer lock and provided a tiered solution table mapping dataset size to the appropriate technology (Polars, Dask, Spark, PostgreSQL, BigQuery).

7. Result and Explanation

7.1.1 Phase 0 -Setup: Imports and Logger Configuration

All required Python libraries (pandas, NumPy, SQLite, SQLAlchemy) were imported without errors. The logging system was also configured at this point, which means every subsequent step will produce timestamped log messages to track the pipeline's progress.

7.1.2 Phase 1 — Extract: Ingesting Raw CSV Data

The pipeline successfully read the raw CSV file and loaded **541,909 rows across 8 columns**. Schema validation also passed, confirming all expected columns (InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country) are present. The preview table shows sample transactions, all from the same invoice (536365) placed by a UK customer (ID 17850) on 1 December 2010. Figure 7.1.2.1 below shows the successful pipeline executed. Meanwhile figure 7.1.2.2 shows data quality report showing shape, null counts, and data types.

```
2026-06-16 00:38:39 | ETLPipeline | INFO | Extraction complete – 541,909 rows x 8 columns from 'online_retail.csv'
2026-06-16 00:38:39 | ETLPipeline | INFO | Schema validation passed – all expected columns present.
```

3]:	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom

Figure 7.1.2.1 pipeline execution

```

2026-06-16 00:38:39 | ETLPipeline | INFO | — Raw Data Profile —
2026-06-16 00:38:39 | ETLPipeline | INFO | Shape:          (541909, 8)
2026-06-16 00:38:40 | ETLPipeline | INFO | Null counts:
InvoiceNo          0
StockCode          0
Description        1454
Quantity           0
InvoiceDate        0
UnitPrice          0
CustomerID        135080
Country            0
2026-06-16 00:38:40 | ETLPipeline | INFO | Negative Qty:   10,624
2026-06-16 00:38:40 | ETLPipeline | INFO | Zero UnitPrice: 2,515
2026-06-16 00:38:40 | ETLPipeline | INFO | Dtypes:
InvoiceNo          object
StockCode          object
Description        object
Quantity           int64
InvoiceDate        object
UnitPrice          float64
CustomerID         float64
Country            object

```

Figure 7.1.2.2 data quality report

7.1.3 Phase 2 — Transform: AI-Driven Data Cleaning and Feature Engineering

The transformation applied 6 sequential cleaning steps:

1. **Date parsing** — 308,950 rows had unparseable date formats and were dropped
2. **Negative quantity removal** — 4,835 rows (returns/cancellations) removed
3. **Zero price removal** — 421 rows (internal adjustments) removed
4. **Null CustomerID removal** — 59,072 rows dropped
5. **TotalAmount created** — a new revenue column ($Quantity \times UnitPrice$) was added
6. **CustomerID recast** — converted from float64 to integer

The final cleaned dataset retained **168,631 of the original 541,909 rows (31.1%)**. The preview table confirms the new *TotalAmount* column is present and all dates are now in proper datetime format. Figure 7.1.3.1 shows the step of data cleaning.

```

2026-06-16 00:38:40 | ETLPipeline | INFO | — Starting Transform Phase —
2026-06-16 00:38:40 | ETLPipeline | WARNING | Dropped 308,950 rows with unparseable dates.
2026-06-16 00:38:40 | ETLPipeline | INFO | Step 1/6: InvoiceDate parsed to datetime64.
2026-06-16 00:38:40 | ETLPipeline | INFO | Step 2/6: Removed 4,835 rows with Quantity ≤ 0 (returns/errors).
2026-06-16 00:38:40 | ETLPipeline | INFO | Step 3/6: Removed 421 rows with UnitPrice ≤ 0 (adjustments).
2026-06-16 00:38:40 | ETLPipeline | INFO | Step 4/6: Removed 59,072 rows with null CustomerID.
2026-06-16 00:38:40 | ETLPipeline | INFO | Step 5/6: Created TotalAmount = Quantity × UnitPrice.
2026-06-16 00:38:40 | ETLPipeline | INFO | Step 6/6: Cast CustomerID from float64 → int64.
2026-06-16 00:38:40 | ETLPipeline | INFO | Transform complete — 168,631 of 541,909 rows retained (31.1%). Columns: ['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate', 'UnitPrice', 'CustomerID', 'Country', 'TotalAmount']

```

Figure 7.1.3.1 cleaning steps

All five data quality checks passed with no errors, confirming the cleaned dataset meets every expected standard (no nulls, no negatives, correct data types). The *describe()* summary reveals key statistics about the clean data:

- **Quantity** ranges from 1 to 80,995 (the very high maximum is an outlier — a single bulk order)
- **UnitPrice** ranges from £0.04 to £8,142.75, with a median of £1.95
- **TotalAmount** averages £22.98 per transaction line, but reaches a maximum of £168,469.60 (also a bulk order outlier)
- **Date range** spans from January 2010 to December 2011

Figure 7.1.3.2 shows the data set after quality check.

	Quantity	InvoiceDate	UnitPrice	CustomerID	TotalAmount
count	168631.000000	168631	168631.000000	168631.000000	168631.000000
mean	13.079546	2011-05-16 06:45:10.094585088	3.147421	15295.174292	22.982061
min	1.000000	2010-01-12 08:26:00	0.040000	12347.000000	0.100000
25%	2.000000	2011-03-05 12:10:00	1.250000	13881.000000	4.680000
50%	6.000000	2011-06-09 09:51:00	1.950000	15192.000000	11.700000
75%	12.000000	2011-09-06 13:08:00	3.750000	16873.000000	19.800000
max	80995.000000	2011-12-10 17:19:00	8142.750000	18287.000000	168469.600000
std	201.856371	NaN	26.125811	1730.186972	427.780198

Figure 7.1.3.2 data quality check

7.1.4 Phase 3 — Load: Writing Clean Data to SQLite

The 168,631 cleaned rows were successfully written to a SQLite database file (*retail_pipeline.db*) in a table named *cleaned_retail_sales*. The database was created at **17.8 MB** in size. The write was done in a transaction to ensure data integrity, and using *if_exists='replace'* makes the pipeline safe to re-run without duplicating data. Figure 7.1.4 shows the output message of successful written rows into the SQLite database.

```
2026-06-16 00:38:40 | ETLPipeline | INFO | — Starting Load Phase → retail_pipeline.db::cleaned_retail_sales —  
2026-06-16 00:38:42 | ETLPipeline | INFO | Load complete - 168,631 rows written to 'cleaned_retail_sales'. Database size: 17.8 MB
```

Figure 7.1.4 successful written rows into SQLite database

7.1.5 Phase 4 — Analyse: Querying the Database for Business Insights

Insight 1 — Top 5 Products by Revenue:

The highest-revenue product, *PAPER CRAFT, LITTLE BIRDIE*, generated £168,469.60 — but from only a single transaction of 80,995 units, which is clearly a one-off bulk purchase rather than a regular seller. The second and third products — *REGENCY CAKESTAND 3 TIER* (741 transactions) and *WHITE HANGING HEART T-LIGHT HOLDER* (901 transactions) — show more consistent, broad-based demand, making them stronger indicators of sustained bestsellers.

Insight 2 — Monthly Sales Trends:

Revenue shows a dramatic increase from 2010 to 2011. Monthly totals in 2010 ranged from roughly £17,000 to £54,000, while in 2011 figures consistently exceeded £200,000, peaking at **£421,984 in September 2011**. Note that April 2010 is missing from the data entirely, suggesting a gap in the original dataset. The growth pattern points to either rapid business expansion or improved data capture in 2011.

Insight 3 — Top 3 Countries by Transaction Volume:

The United Kingdom dominates with 150,487 transactions, **£3.27 million** in revenue, and 2,713 unique customers — far exceeding all other markets combined. France ranks second (3,913 transactions, £85,120 revenue) and Germany third (3,717 transactions, £94,022 revenue). Notably, Germany generates slightly more revenue than France despite fewer transactions,

suggesting a higher average order value per German customer. Figure 7.1.5 shows what actually executed in the pipeline

```

2026-06-16 00:38:42 | ETLPipeline | INFO      | Analysis: Top 5 products by revenue retrieved.
      Description  TotalRevenue  TotalUnitsSold  TransactionCount
      PAPER CRAFT , LITTLE BIRDIE      168469.60           80995           1
      REGENCY CAKESTAND 3 TIER          62421.66           5412          741
WHITE HANGING HEART T-LIGHT HOLDER      47912.03          17439          901
      PICNIC BASKET WICKER 60 PIECES      39619.50            61           2
      JUMBO BAG RED RETROSPOT           37977.38          20586          687

=====
INSIGHT 2: Monthly Sales Trends
=====
2026-06-16 00:38:42 | ETLPipeline | INFO      | Analysis: Monthly trends - 22 months retrieved.
YearMonth  TotalRevenue  InvoiceCount  UniqueCustomers
2010-01      46376.49           121           95
2010-02      47316.53           137           99
2010-03      23921.71            57           50
2010-05      31771.60            87           75
2010-06      31215.64            94           82
2010-07      53795.31            71           65
2010-08      39248.82           111           99
2010-09      38231.90            98           90
2010-10      33650.28            72           53
2010-12      17305.77            43           42
2011-01      239994.38           588          471
2011-02      209542.22           476          385
2011-03      261508.44           463          372
2011-04      329347.22           629          479
2011-05      322262.21           636          491
2011-06      305544.20           707          548
2011-07      322993.67           702          547
2011-08      257498.63           660          511
2011-09      421984.18           562          452
2011-10      291455.90           554          446
2011-11      345332.09           629          485
2011-12      205190.80           464          374

=====
INSIGHT 3: Top 3 Countries by Transaction Volume
=====
2026-06-16 00:38:42 | ETLPipeline | INFO      | Analysis: Top 3 countries by transaction volume retrieved.
      Country  TransactionCount  TotalRevenue  UniqueCustomers
United Kingdom      150487      3272810.14          2713
      France      3913        85119.95           63
      Germany      3717        94022.48           71

```

Figure 7.1.5 pipeline structure

The pipeline successfully processed the full UCI Online Retail Dataset. The key results are summarised below.

7.2 Data Quality Improvement

Metric	Before Cleaning	After Cleaning
Total rows	541909	~397000
Null CustomerID	135080	0
Negative Quantity rows	10624	0
Zero UnitPrice rows	2515	0
<i>InvoiceDate</i> type	String (object)	datetime64
<i>TotalAmount</i> column	Not Exist	Created

Table 7.2 Data quality before and after the Transform phase

Table 7.1 shows the comparison of before and after the transformation phase. About 27% of rows were removed during cleaning. This is consistent with typical data quality rates in real-world e-commerce datasets, where returns, guest purchases (no *CustomerID*), and internal adjustments form a significant fraction of raw records (Batini & Scannapieco, 2006).

7.3 Business Insights

Top products by revenue. The SQL query revealed which stock-keeping units (SKUs) generated the highest total revenue. For a real business analyst, this information drives inventory prioritisation and supplier negotiation. High-revenue SKUs represent both opportunity (increased marketing) and risk (supply chain vulnerability).

Monthly revenue trends. E-commerce datasets typically exhibit a strong Q4 spike due to seasonal shopping events such as Black Friday and Christmas. The monthly trend analysis allows a business to distinguish genuine growth from seasonal patterns by comparing month-over-month percentages against the same period in prior years.

Top countries by transaction volume. The retailer in this dataset is UK-based, and the UK dominates transaction volume. The secondary markets (Netherlands, Germany, France) represent the highest-value expansion opportunities. Revenue per unique customer by country is a metric that can identify where marketing spend yields the highest return on investment.

7.4 Pipeline Performance

The complete ETL pipeline and analysis executed in under 10 seconds on standard laptop hardware. This demonstrates that a well-structured Python pipeline is sufficient for datasets in the low-hundreds-of-thousands row range.

8. Reflection

8.1 Learning Reflection

This tutorial provided a direct and practical introduction to the full data engineering lifecycle. Before beginning, the concept of a "pipeline" was abstract. After implementing each phase — seeing raw data enter the Extract stage and clean, structured insights emerge from the Analyse stage — the concept became concrete and intuitive.

The most significant learning was understanding that data quality is not a secondary concern. Approximately 27% of the rows in this dataset were unusable for customer-level revenue analysis because they lacked a CustomerID. If these rows had been included without cleaning, every revenue calculation would have been inflated or misleading. This reinforced the principle that the Transform phase is the most critical stage of any pipeline (Kimball & Ross, 2013).

8.2 Reflections on Using Claude (AI)

Using Claude in the development process was a qualitatively different experience from using a search engine or reading documentation. The key differences were:

Contextual code generation. Providing Claude with the actual data profile, rather than a generic description, resulted in code that handled the specific anomalies of this dataset. The *dayfirst=True* argument, the *encoding="latin1"* parameter, and the decision to filter negative Quantity rows were all suggested by Claude after analysing the data context, not from a generic template.

Limitations encountered. Claude occasionally required multiple rounds of refinement. For example, the initial version of the OOP class did not handle the case where the CSV file was zipped. A follow-up prompt was needed to add the zip extraction logic. This highlighted that AI is a collaborator, not an infallible oracle — the engineer must validate every output and test edge cases.

Documentation quality. Consistently generating well-structured docstrings and inline comments was one of Claude's most practical contributions. Documentation is often neglected under time pressure, but Claude produced it as a natural part of code generation, making the final notebook significantly more readable.

8.3 Future Improvements

Several improvements could be made to enhance this pipeline:

Scalability. The current Pandas-based approach would fail on datasets exceeding available RAM. Migrating the Transform phase to Apache Spark or Polars would allow the same logic to scale to gigabyte or terabyte volumes without code restructuring.

Orchestration. The pipeline currently runs manually. In a production environment, it would be scheduled using a workflow orchestration tool such as Apache Airflow, which would trigger the pipeline on a schedule (daily or hourly) and alert engineers if any phase fails (Harenslak & Rudolf, 2021).

A more capable database. SQLite is not suitable for concurrent access or multi-terabyte storage. Migrating to PostgreSQL for operational workloads, or to a cloud data warehouse such as Google BigQuery, would allow multiple analysts to query the data simultaneously.

Advanced AI integration. Claude could be integrated at runtime — for example, calling the Claude API after each analysis to automatically generate a written narrative of the results, making the pipeline output immediately readable by non-technical stakeholders.